CHALLENGE SHEET

Dual Check

Two independent Windows PE binaries bound by a shared secret — recover an obfuscated password from game.exe, then reimplement a chained transform read from checker.exe's decompiled code to derive the AES-128 key that decrypts the embedded flag.

DIFFICULTY	TRACK	HOST PORT	LAB ID
MEDIUM	BINARY	8091	dual-check

EASY · MEDIUM · HARD · INSANE

VERIFY THE SOLVE

```
• python3 ../scripts/check.py dual-check
```

Brief

Two independent Windows PE binaries, game.exe and checker.exe, bound by a shared secret. Recover an obfuscated password from game.exe, then read checker.exe's decompiled code to reimplement a chained byte transform and derive the AES-128 key that decrypts the embedded flag.

Setup

No install step. Run the binaries directly:

```
wine bin/game.exe      # or run natively on Windows
wine bin/checker.exe
```

No server-side component and no network dependency — this is an artifact-only BYOD lab. See "Delivery model (artifact-only exception)" below for why docker-compose.yml exists anyway.

Goal

Recover both passwords and the derived AES key through static analysis (a decompiler such as Ghidra), then run checker.exe to decrypt and print the flag. Verify with:

```
python3 check.py "duck\{...\}"
```

What this is

A custom-built, two-binary reverse engineering crackme (`game.exe` + `checker.exe`), designed and built from scratch for this task, then solved end-to-end to validate it and produce the evidence below.

Delivery model (artifact-only exception)

This lab has **no HTTP service the solve depends on**. The player is handed two Windows PE executables (`bin/game.exe`, `bin/checker.exe`) and runs them directly — natively on Windows, or under Wine on Linux/macOS. No server-side state exists: all checking happens inside the binaries themselves (`checker.exe`'s own password-hash and flag-decryption logic), plus the player-facing `check.py` at the lab root, which only checks a candidate flag against this directory's `lab.yml` — it does not run or replace the binaries.

`docker-compose.yml` and `lab.yml`'s `port: 8091` exist only to satisfy the catalog's general `docker-compose.yml/validate.py` requirement — they run a distribution-only HTTP server for the two binaries and the docs, and nothing in the intended solve path ever touches that service.

Reset / state model: not applicable. Neither binary has any persistent or server-side state, and neither password nor the flag rotates per playthrough — every player who solves it correctly gets the same, fixed flag. There is nothing to reset between attempts; running the binaries again is itself the reset.

Platform: Windows PE only, by design — both binaries are compiled with MinGW/TDM-GCC targeting Windows, and `game.c` uses Windows-only APIs (`<windows.h>`) for console behavior. No Linux ELF build is provided, so no headless-checker CI step applies here; see `src/BUILD_INSTRUCTIONS.md` for the exact build command if a maintainer wants to rebuild from source.

Repo layout

```
.
│
├── bin/                          ← the compiled, playable challenge files
│   ├── game.exe
│   └── checker.exe
│
├── docker-compose.yml            ← distribution-only helper (see "Delivery model")
├── lab.yml                       ← catalog metadata (track, difficulty, flag_hash,
techniques)
├── check.py                      ← player-facing flag checker (reads lab.yml, no deps)
│
├── docs/                          ← all written evidence/report + maintainer docs
│   ├── README.md
│   ├── player-briefing.mdx       ← player-facing description (no spoilers)
│   ├── challenge-design.mdx
│   ├── official-solution.mdx     ← full spoiler solve, including the narrated,
│   │                                screenshot-illustrated walkthrough
│   ├── security-report.mdx
│   ├── remediation.mdx
│   ├── validation-report.mdx
│   └── images/                   ← screenshots referenced from official-solution.mdx
│
└── src/                           ← full source code (private repo only — never ship
to participants)
    ├── BUILD_INSTRUCTIONS.md     ← how to compile game.exe / checker.exe from source
    ├── game.c
    ├── checker.c
    ├── aes.c / aes.h
    ├── base64.c / base64.h
```

```
        ├── sha256.c / sha256.h
        ├── test_sha256.c            ← dev-only: validates the SHA-256 implementation
        └── gen_checker_v3.c         ← dev-only: generates the embedded constants in
    checker.c
```

Where to start

If you want to…	Go to
Understand what the challenge is testing	`docs/challenge-design.mdx` (”Skills exercised”)
See the player-facing description (no spoilers)	`docs/player-briefing.mdx`
Reproduce the full solve, step by step, with screenshots	`docs/official-solution.mdx` (”Full narrated walkthrough”)
Understand the security root cause being demonstrated	`docs/security-report.mdx`
See mitigation recommendations for real-world systems	`docs/remediation.mdx`
Rebuild the binaries from source	`src/BUILD_INSTRUCTIONS.md`
Check a candidate flag	`check.py "duck{...}"`
See catalog metadata (track, difficulty, flag hash)	`lab.yml`
See what’s verified vs. still pending	`docs/validation-report.mdx`

Design summary

- `game.exe` and `checker.exe` are independent binaries — `checker.exe` never reads anything from `game.exe` at runtime.
- `game.exe`: recover an obfuscated password (multi-byte XOR + Base64 + byte-masking) → pass a knowledge quiz about that analysis → win a mini-game → receive a second, independently-obfuscated password.
- `checker.exe`: requires (1) that second password, verified only via a stored SHA-256 hash, and (2) a 16-byte value that is **not stored anywhere in the binary** — it must be computed by reading and replicating a custom chained transform found in the decompiled code. Only when both are correct does the program derive the final AES-128 key, decrypt the embedded flag, and print it.
- Neither binary leaks the flag, the flag’s format, or either password to a plain `strings` scan.

Evidence status

- x Challenge designed and built (C, compiled with MinGW/TDM-GCC)
- x Full solve performed and verified (Ghidra static analysis + Python/CyberChef reproduction)
- x Flag confirmed reproducible via two independent paths (running `checker.exe`, and manual offline decryption)
- x `strings` scan confirms no leakage of flag/passwords on either binary
- x Full walkthrough, objective/skills, player briefing, root-cause and mitigation documentation completed (all consolidated under `docs/` — see `docs/README.md`)
- x Flag regenerated to be fully random and catalog-format-compliant (`^duck\{[a-z0-9_`

]{16,40}\}$); lab.yml created with its SHA-256 flag_hash

- x checker.c's embedded flag ciphertext updated to match — password, hex-value algorithm, and game.exe are all byte-for-byte unchanged, so the documented walkthrough solve path still applies exactly as written
- x Chain logic re-verified end-to-end on a Linux-compiled test build of the patched checker.c (same documented password + hex value → new flag; wrong password and wrong hex value both still correctly rejected)
- x **checker.exe rebuilt from the updated checker.c** on a Windows machine with MinGW/TDM-GCC and pushed — independently re-confirmed by extracting the current FLAG_IV/FLAG_CIPHERTEXT byte arrays from src/checker.c and finding those exact bytes embedded in the committed bin/checker.exe at the expected offsets (see docs/validation-report.mdx's evidence ledger). A live interactive re-solve of this exact binary is still recommended but is no longer a blocker.
- x check.py added at the lab root; verifies a candidate flag against lab.yml's flag_hash, no dependencies beyond the standard library